

Assignment 2 – Case Study

Stage 4 Tutorial Activities

Object-Oriented Design Decisions

Week 7 | 60 minutes

Activity 1 - Encapsulation Review

Class	Protected state / invariant	Public operations
Patient	Protected Attributes: _patient_id, _full_name _phone, _dob _address _appointment_history Invariants: patient_id _appointment_history	update_details(phone, address), add_to_history(appointment), get_appointment_history() Read-only properties (@property): patient_id, full_name, phone, dob, address
Practitioner	Protected Attributes: _practitioner_id, _doctor_name, _speciality, _room_number, _assigned_appointments Invariants: _practitioner_id is immutable once instantiated. _assigned_appointments	assign_appointment(appointment) get_daily_schedule(date) Read-only properties (@property): practitioner_id, doctor_name, speciality, room_number
Appointment	Protected Attributes: _appointment_id _patient_id, _practitioner_id _date_time, _status _cancellation_reason Invariants: _status	create_booking(...) (factory method) check_collision(...) (static validation) update_status(new_status) cancel_appointment(reason) is_active()

Activity 2 - Composition or Inheritance?

Appointment and Patient -> ☒ Composition/association ☐ Inheritance Reason: An Appointment and a Patient are independent real-world entities with separate lifespans.

Appointment and Practitioner -> ☒ Composition/association ☐ Inheritance Reason: An Appointment references the Practitioner who conducts the consultation.

Doctor and Practitioner (hypothetical) -> ☐ Composition/association ☒ Inheritance Reason: A Doctor (or General Practitioner, Cardiologist) is a specific type of Practitioner

Clinic and Appointment -> ☒ Composition/association ☐ Inheritance Reason: A Clinic (or ClinicController) acts as an overarching container or registry that holds and manages collections of Appointment objects.

Activity 3 - Responsibility Allocation

Who decides whether SCHEDULED can become CANCELLED?

ANSWER: The Appointment class (specifically its state transition logic).

Who validates a patient name?

ANSWER: The Patient class upon instantiation/update, and optionally the ClinicController as an application guardrail.

Should Appointment execute SQL? Why?

ANSWER: Database interaction (SQL/ORM) belongs in a dedicated Persistence Layer / Repository (e.g., AppointmentRepository), keeping the domain model clean, portable, and fast.

Should the UI decide whether a status transition is legal?

ANSWER: No. The UI's job is presentation and capturing user intent. The domain layer (Appointment) is the single source of truth that enforces whether a requested transition is legal.

Activity 4 - AI Code Critique

AI generates an Appointment class with public status mutation, SQL inside cancel(), a NotificationManager dependency and inheritance from PatientRecord. Identify at least five design problems and corrections.

PROBLEM: The UML models an Appointment as being associated with a Patient, not a type of Patient.

CORRECTION: Use association via patient_id and practitioner_id

PROBLEM: Public status mutation bypasses lifecycle rules and cancellation validation.

CORRECTION: Private _status + controlled methods

PROBLEM: SQL inside cancel() violates separation of responsibilities.

CORRECTION: Keep persistence outside the class

PROBLEM: NotificationManager dependency, Appointment becomes tightly coupled to infrastructure.

CORRECTION: Remove the dependency entirely.

PROBLEM: Allows invalid values such as: appointment.status = "Done"

CORRECTION: Use an enum.

Exit question

Why can code be object-oriented syntactically but still have poor object-oriented design?

ANSWER: Syntactic object-orientation is about structure, whereas good OOD is about responsibility, encapsulation, and behavior.